Course Name:	Artificial Intelligence Laboratory	Semester:	VI
Date of Performance:	_27__ / _01__ / _2026_____	DIV/ Batch No:	C-3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Experiment No: 1

Title: Implementation of PROLOG programs

Aim: To implement condition-action rules based agent and Goal based agent using PROLOG program

Objectives of the Experiment:

By the end of this experiment, students will be able to:

- Understand the basics of **logic programming** using Prolog.
- Represent real-world relationships using **facts and rules**.
- Apply **logical inference and backtracking** to answer relationship queries.
- Write and execute Prolog programs to derive **implicit.**

COs to be achieved:

CO1: Design AI solution with appropriate choice of agent architecture

Books/ Journals/ Websites referred:

1. *https://swish.swi-prolog.org/*
2. *https://www.tutorialspoint.com/prolog/prolog_introduction.htm*

Theory:

Agent programs for simple applications need not be very complicated. They can be based on condition-action rules and still they give better results, though not always rational. The family tree program makes use of similar concept.

A simple agent program can be defined mathematically as an agent function which maps every possible percepts sequence to a possible action the agent can perform or to a coefficient, feedback element, function or constant that affects eventual actions:

$$F: P^* -> A$$

Algorithm for 'Condition-Action Rule Table' Agent function:

functionSIMPLE-REFLEX-AGENT (percept) **returns** an action
Static: *rules,* a set of condition-action rules
State:- INTERPRET-INPUT (percept)
Rule:- *RULE-MATCH (state, rules)*
Action:- *RULE-ACTION [rule]*
Returnaction

This approach follows a table for lookup of condition-action pairs defining all possible condition-action rules necessary to interact in an environment.

Goal Based Agent:

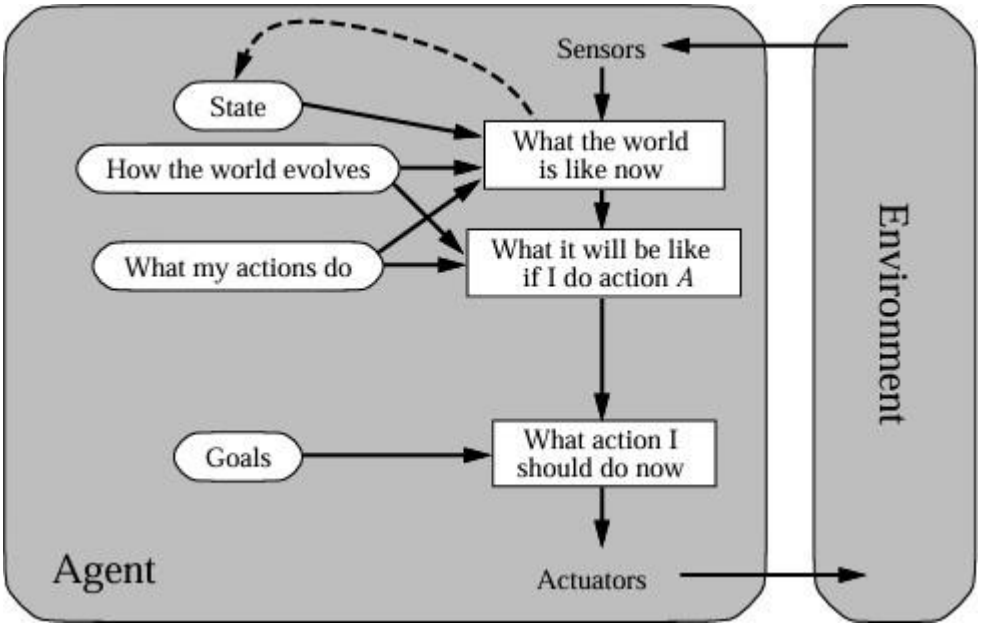
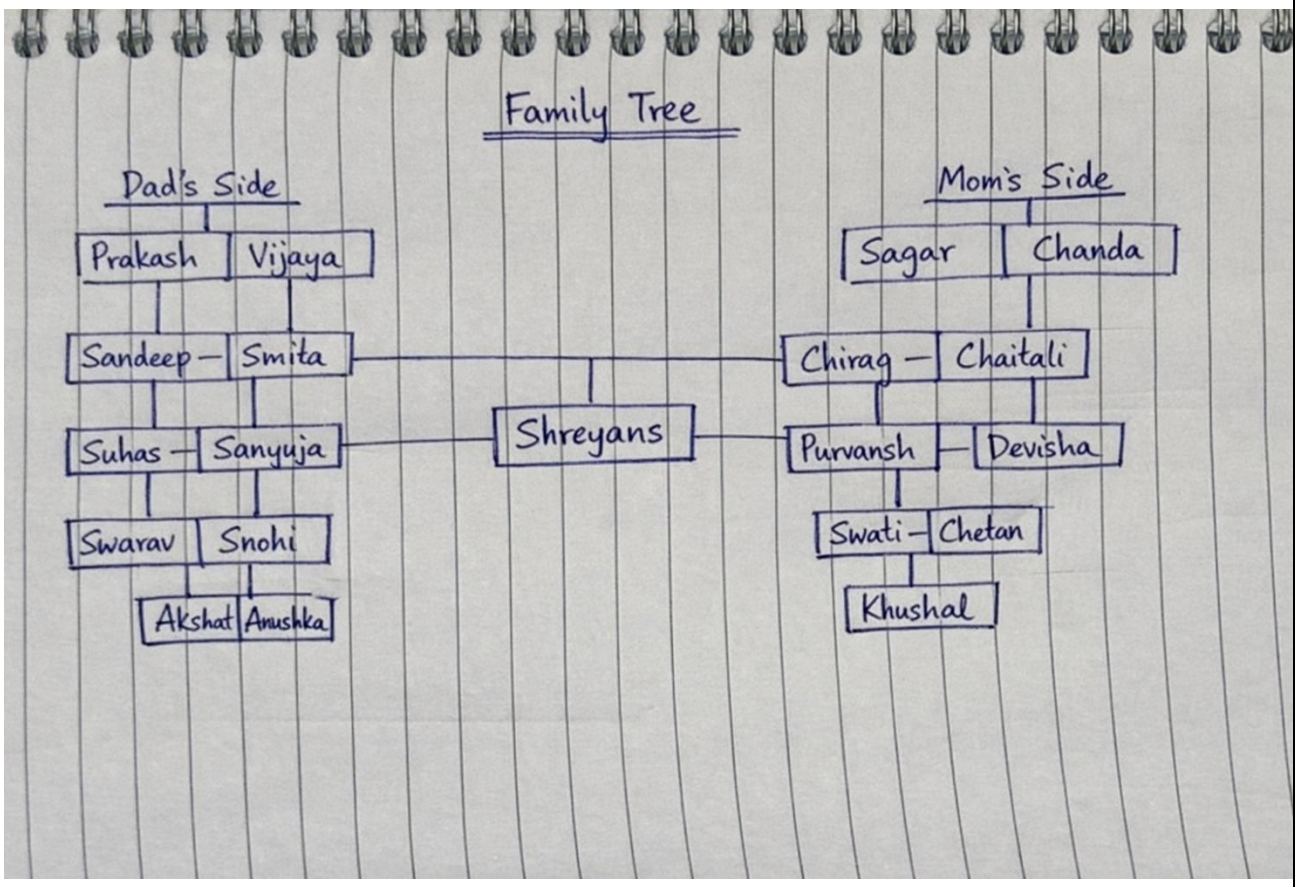


Figure: Goal based agent architecture

The decision making in goal based agents is fundamentally different from the condition-action rules based agents. A goal-based agent, in principle, could reason that if the car in front has its brake lights on, it will slow down. Given the way the world usually evolves, the only action that will achieve the goal of not hitting other cars is to brake. Although the goal-based agent appears less efficient, it is more flexible because the knowledge that supports its decisions is represented explicitly and can be modified.

Activity 1: Example Family Tree/disease-symptom mapping/Rule-based chatbot/City map with their distances between them:



Code:

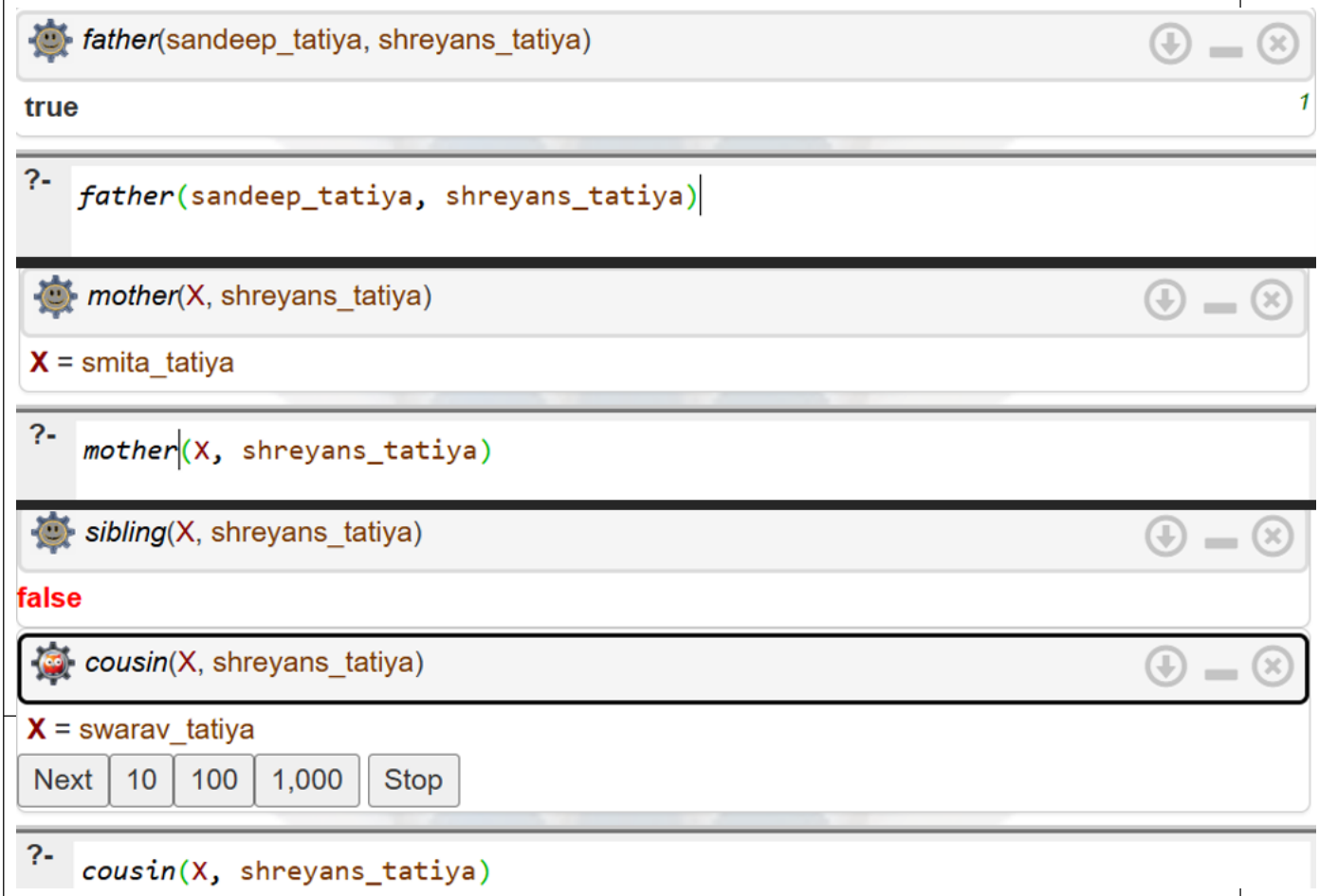
```
% ===== Dad Side =====
parent(prakash_tatiya, sandeep_tatiya).
parent(vijaya_tatiya, sandeep_tatiya).

parent(prakash_tatiya, suhas_tatiya).
parent(vijaya_tatiya, suhas_tatiya).
```

```
parent(prakash_tatiya, neeta_zabak).
parent(vijaya_tatiya, neeta_zabak).

% Sandeep + Smita -> Shreyans
parent(sandeep_tatiya, shreyans_tatiya).
parent(smita_tatiya, shreyans_tatiya).

% Suhas + Sanyuja -> Swarav, Snohi
parent(suhas_tatiya, swarav_tatiya).
parent(sanyuja_tatiya, swarav_tatiya).

parent(suhas_tatiya, snohi_tatiya).
parent(sanyuja_tatiya, snohi_tatiya).

% Neeta + Nitin -> Akshat, Anushka
parent(neeta_zabak, akshat_zabak).
parent(nitin_zabak, akshat_zabak).

parent(neeta_zabak, anushka_zabak).
parent(nitin_zabak, anushka_zabak).

% ===== Mom Side =====
parent(sagar_jain, smita_tatiya).
parent(chanda_jain, smita_tatiya).

parent(sagar_jain, chirag_jain).
parent(chanda_jain, chirag_jain).

parent(sagar_jain, swati_nahar).
parent(chanda_jain, swati_nahar).

% Chirag + Chaitali -> Purvansh, Devisha
parent(chirag_jain, purvansh_jain).
parent(chaitali_jain, purvansh_jain).

parent(chirag_jain, devisha_jain).
parent(chaitali_jain, devisha_jain).

% Swati + Chetan -> Khushal
parent(swati_nahar, khushal_nahar).
parent(chetan_nahar, khushal_nahar).

% Gender Classification
```

```
male(prakash_tatiya).
male(sandeep_tatiya).
male(suhas_tatiya).
male(nitin_zabak).
male(swarav_tatiya).
male(akshat_zabak).
male(sagar_jain).
male(chirag_jain).
male(chetan_nahar).
male(purvansh_jain).
male(khushal_nahar).
male(shreyans_tatiya).

female(vijaya_tatiya).
female(smita_tatiya).
female(sanyuja_tatiya).
female(neeta_zabak).
female(snohi_tatiya).
female(anushka_zabak).
female(chanda_jain).
female(chaitali_jain).
female(swati_nahar).
female(devisha_jain).

% Father / Mother
father(X, Y) :- parent(X, Y), male(X).
mother(X, Y) :- parent(X, Y), female(X).

% Grandparent
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
grandfather(X, Z) :- grandparent(X, Z), male(X).
grandmother(X, Z) :- grandparent(X, Z), female(X).

% Sibling (same parent)
sibling(X, Y) :- parent(P, X), parent(P, Y), X \= Y.
brother(X, Y) :- sibling(X, Y), male(X).
sister(X, Y)  :- sibling(X, Y), female(X).

% Uncle / Aunt
uncle(U, C) :- male(U), sibling(U, P), parent(P, C).
aunt(A, C)  :- female(A), sibling(A, P), parent(P, C).

% Cousin
cousin(X, Y) :-
```

```
parent(P1, X),
parent(P2, Y),
sibling(P1, P2),
X \= Y.
```

Output:


father(sandeep_tatiya, shreyans_tatiya)
true
1
?- father(sandeep_tatiya, shreyans_tatiya)
mother(X, shreyans_tatiya)
X = smita_tatiya
?- mother(X, shreyans_tatiya)
sibling(X, shreyans_tatiya)
false
cousin(X, shreyans_tatiya)
X = swarav_tatiya
Next
10
100
1,000
Stop
?- cousin(X, shreyans_tatiya)

Activity 2 : Implement a goal-based agent using a PROLOG program (Crime Mystery)

Problem statement: The Blackout Packet

A murder has occurred at **CipherByte Labs** during a late-night system upgrade.
Aarav Mehta was found dead in the **server control room**, which was locked and monitored.

There are nine people involved in the case:

- One victim
- One perpetrator
- Four suspects
- Three witnesses

The murder weapon is believed to be either a **malware kill-switch** or a **rigged smart UPS trigger** (power spike attack).

The suspects had varying levels of access, motive, and opportunity.
The goal is to use Prolog facts and logical rules to determine who the murderer is based on:

- motive
- access to the crime scene
- access to the weapon
- absence of alibi

```
% -------- PEOPLE --------

victim(aarav).

suspect(vikram). suspect(isha). suspect(ceo_malhotra). suspect(it_admin).

witness(isha). witness(security_guard). witness(forensics_expert).

% -------- WEAPONS --------

weapon(malware). weapon(ups_trigger).
```

```
% -------- MOTIVE --------

motive(vikram, data_extortion).

% -------- ACCESS --------

access(vikram, control_room). access(vikram, malware).

access(isha, control_room). access(it_admin, control_room).

% -------- ALIBI --------

alibi(isha). alibi(it_admin). alibi(ceo_malhotra).

% -------- RULES --------

has_motive(X) :- motive(X, _).

has_access(X) :- access(X, control_room), access(X, malware).

is_suspect(X) :- suspect(X).

killer(X) :- is_suspect(X), has_motive(X), has_access(X),
          \+ alibi(X).
```

Output:

killer(X).

X = vikram

Next | 10 | 100 | 1,000 | Stop

```
?- killer(X).
```

has_motive(X).

X = vikram

```
?- has_motive(X).
```

has_access(X).

X = vikram

Next | 10 | 100 | 1,000 | Stop

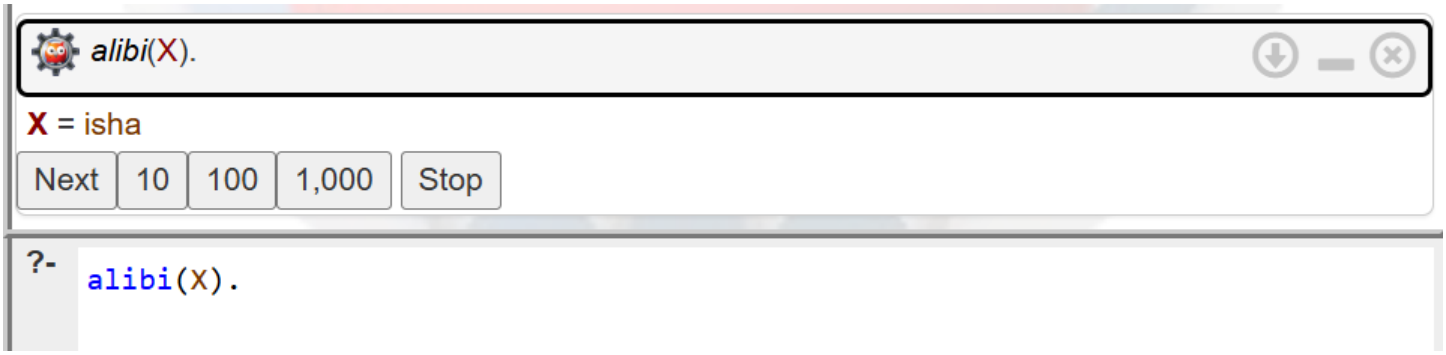
```
?- has_access(X).
```

is_suspect(X).

X = vikram

Next | 10 | 100 | 1,000 | Stop

```
?- is_suspect(X).
```


alibi(X).
X = isha
Next
10
100
1,000
Stop
?-
alibi(X).

Post Lab Subjective/Objective type Questions:

Question: Answer in brief

1. **Differentiate between a fact and a predicate with syntax.**

Aspect	Fact	Predicate
Meaning	A fact is a **statement that is always true** in the knowledge base	A predicate is a **general relationship or property** that can be true or false
Nature	Specific information	General rule/relationship
Variables	No variables	Contains variables
Usage	Represents concrete data	Used to infer new facts
Example	parent(john, mary).	parent(X, Y)
Syntax	fact_name(constant1, constant2).	predicate_name(variable1, variable2)

2. **Differentiate between knowledgebase and Rule base approach.**

Aspect	Knowledge Base Approach	Rule Base Approach
Content	Stores **facts + rules**	Stores **only rules**
Representation	Declarative knowledge	IF–THEN rules
Flexibility	More expressive	Limited to condition–action
Reasoning	Uses inference mechanisms	Uses rule matching
Example	Prolog knowledge base	Expert systems
Example Rule	father(X,Y) :- parent(X,Y), male(X).	IF fever THEN illness

3. **Differentiate between database and knowledgebase.**

Aspect	Database	Knowledge Base
Purpose	Store and retrieve data	Store and reason with knowledge
Operations	Insert, update, delete	Inference, deduction
Intelligence	No reasoning ability	Has reasoning capability
Structure	Tables and records	Facts, rules, predicates
Example	MySQL, Oracle	Prolog KB, Expert systems

Question: Answer in detail.

Explain the role of PEAS and task environment in choosing the agent architecture. Justify your answer with an example.

PEAS stands for **Performance measure, Environment, Actuators, and Sensors**.
It helps define **what an agent should do** and **how it interacts with the environment**.

The **task environment** is analyzed using properties like observability, determinism, and dynamics. Based on these properties, a suitable **agent architecture** is chosen.

Example: Autonomous Taxi

PEAS:

- **Performance:** Safe and fast travel
- **Environment:** Roads, traffic, pedestrians
- **Actuators:** Steering, brakes
- **Sensors:** Cameras, GPS

Task Environment:
Partially observable, dynamic, and stochastic.

Chosen Agent:
A **utility-based learning agent**, because it can handle uncertainty and improve performance over time.

Conclusion

PEAS defines the task clearly, and task environment analysis helps select the **right type of agent architecture** for efficient performance.

Conclusion:
The experiment demonstrates how Prolog uses **logical inference, unification, and backtracking** to answer queries efficiently. This helps in understanding the declarative nature of Prolog and its effectiveness in solving problems based on relationships and knowledge representation.